

pyop3: An Execution Abstraction for Compact Computational Kernels on Unstructured Meshes

Connor J. Ward
David A. Ham
Jack D. Betteridge

c.ward20@imperial.ac.uk

1. Motivation

The execution of compact computational kernels, where one loops over a mesh and performs some “local” calculation for each mesh entity, is ubiquitous in continuum mechanics simulations. To give an example, in the finite element method one assembles the linear systems to be solved by looping over the cells of the mesh, evaluating an integral calculation using local DoFs, and then adding the result to a global matrix or vector.

As a one-off, this “outer loop” is very straightforward to write by hand. However, there is a plethora of design choices (PDE, discretisation, offload device, etc) and possible optimisations (vectorisation, tiling, data layout transformations, etc) that mean that this loop would need to be rewritten time and time again. This handwritten approach both hampers programmer productivity and inhibits the development of higher level abstractions (e.g. [1]).

Therefore, a software abstraction that captures the semantics and automates the construction of “outer loops” is needed.

2. Introducing pyop3

To this end, we have written a new library for automating the construction and execution of these loops: pyop3. Given some user-provided data structures and a [loop expression](#) containing the application of some local kernel, pyop3 uses [code generation](#) to produce low-level code that is near equivalent to something handwritten (fig. 1).

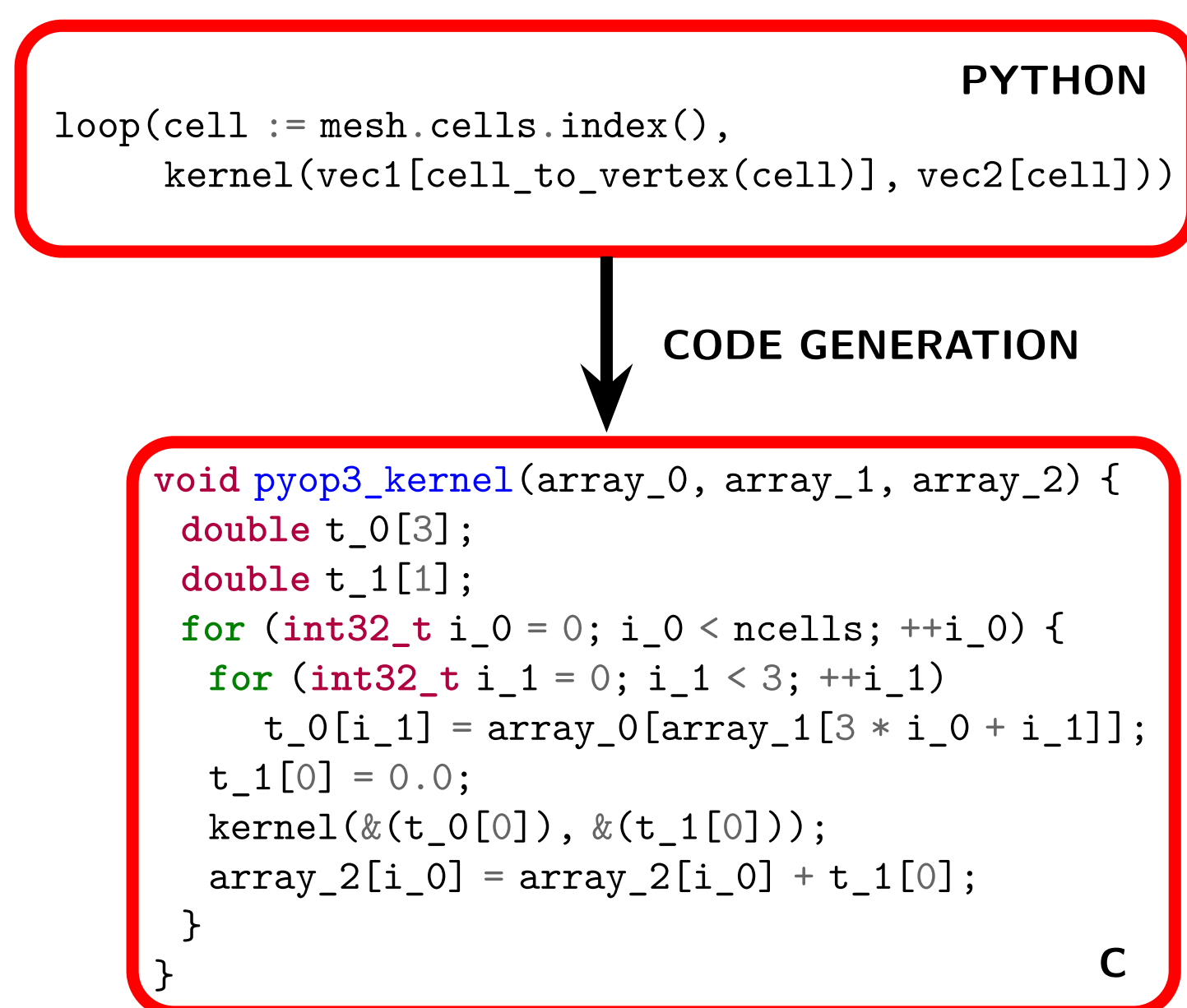


Figure 1: Simplified example of the code generated from a pyop3 loop expression.

pyop3 is similar in many respects to its precursor, PyOP2 [3], and work to replace PyOP2 with pyop3 in the finite element framework Firedrake [2] is ongoing.

References

- [1] Martin S. Alnæs, Anders Logg, et al. “Unified Form Language: A Domain-Specific Language for Weak Formulations of Partial Differential Equations”. In: 40.2 (2014). arXiv: 1211.4047.
- [2] David A. Ham, Paul H. J. Kelly, et al. Firedrake User Manual. First edition. manual. 2023.
- [3] Florian Rathgeber, Graham R. Markall, et al. “PyOP2: A High-Level Framework for Performance-Portable Simulations on Unstructured Meshes”. In: 2012. (Visited on 04/19/2020).

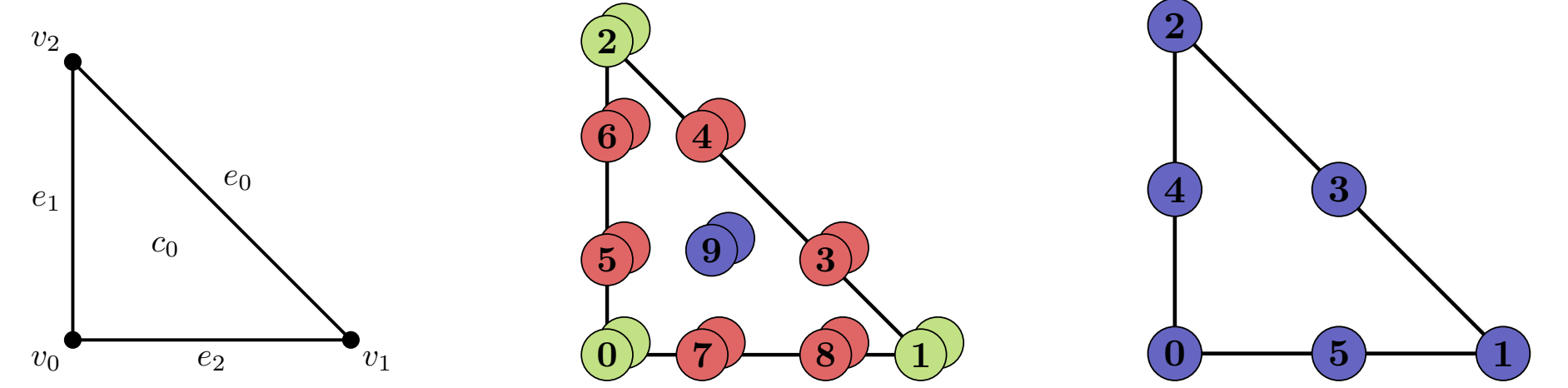


Figure 2: An example of a mixed finite element (Scott-Vogelius, middle and right) on a reference triangle (left). DoFs associated with vertices, edges and cells are shown in green, red and blue respectively. The lower-order element (right) is discontinuous, so all the DoFs are considered to belong to the cell.

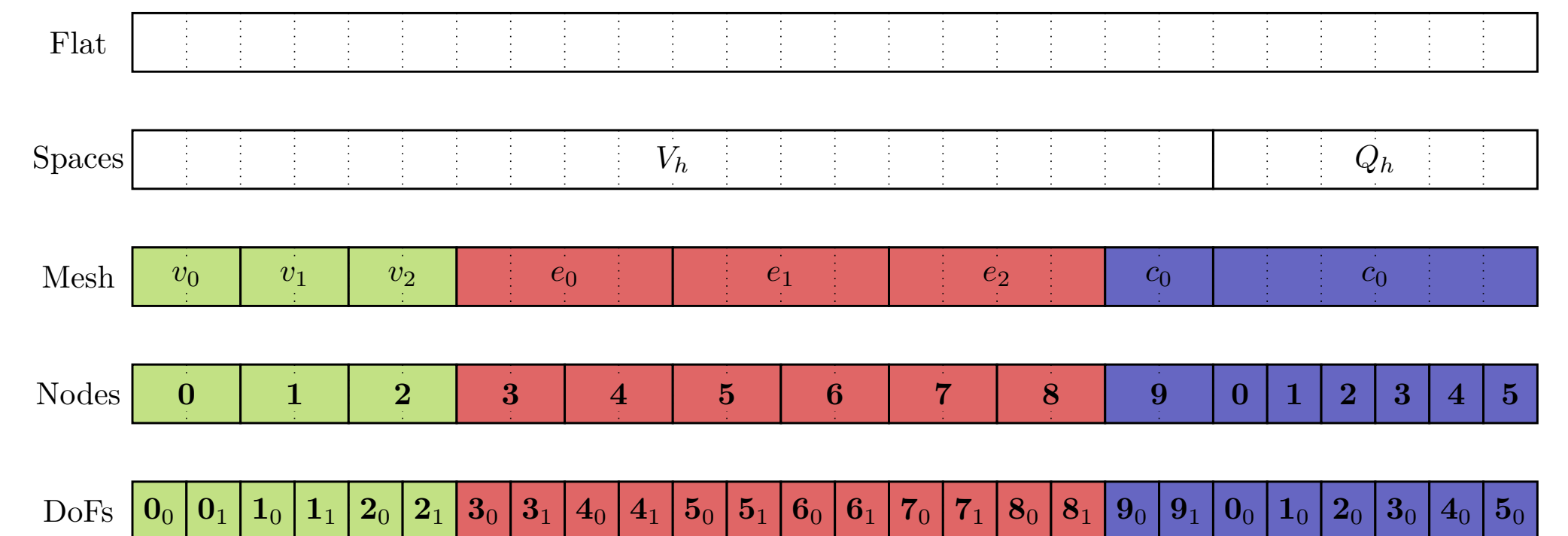


Figure 3: A possible data layout for the DoFs of a one-cell mesh for the finite element pair in fig. 2. The labels V_h and Q_h represent the function spaces of the two elements.

3. A new abstraction for mesh data

The most significant difference between PyOP2 and pyop3 is the development of a new abstraction for describing data layouts. Mesh data has a complicated structure (fig. 3) that maps poorly to existing ways of describing data layouts (e.g. N-dimensional arrays). Whilst PyOP2 handles this structure by throwing away information, pyop3 is capable of fully describing complex mesh data layouts using [axis trees](#) (fig. 4). Having a richer abstraction for data layouts makes composition easier and thus pyop3 is able to describe and generate code for a wider range of problems than PyOP2.

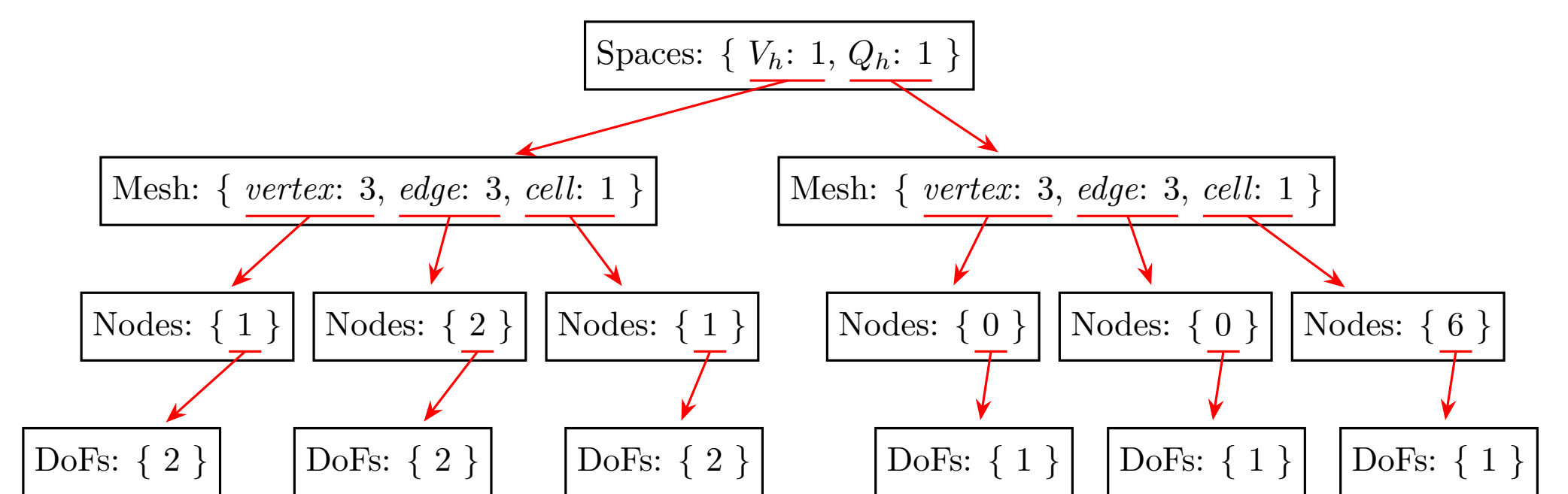


Figure 4: An axis tree that describes the data layout shown in fig. 3.

4. Future work

Once pyop3 has been fully integrated into Firedrake, there is scope for a variety of performance improvements and avenues of research to pursue. Possible low-level changes include:

- GPU support (high priority)
- Data layout transformations
- Loop tiling

Whilst at a high level the following should also become expressible:

- Physics-based preconditioners
- Slope limiters
- Adaptivity
- Particle methods